

Lab 3 Preprocessing with DPLYR

2026-01-15

Lab 3 Preprocessing with DPLYR

Goal

Create a Line Chart to visualize the trend of Trailing 365-Day Distance over time.

Chaining functions

You might see `%>%` in R, this belongs to a package that is part of dplyr along with other useful functions, they are not required, but so commonly used that when searching for solutions you should be aware of them.

To install;

```
install.packages("tidyverse")  
#note if you chose to do this in week 2  
#you already have the required packages.  
#or  
install.packages("dplyr")
```

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
df <- data.frame(  
  name = c("Alice", "Bob", "Charlie", "Alice", "Bob", "Charlie"),  
  score = c(80, 75, 90, 85, 95, 100)  
)  
df
```

```
##      name score  
## 1  Alice    80  
## 2   Bob    75
```

```
## 3 Charlie    90
## 4   Alice    85
## 5     Bob    95
## 6 Charlie   100
```

Note that there are only three names in the above dataframe.

```
summarise(group_by(df, name), avg_score = mean(score))
```

```
## # A tibble: 3 x 2
##   name    avg_score
##   <chr>    <dbl>
## 1 Alice      82.5
## 2 Bob        85
## 3 Charlie    95
```

The above function demonstrates summarise, this is used for collapsing a table into quick summary values, above we have grouped by name, and provided a ‘summary’ of the persons average score.

```
df %>%
  group_by(name) %>%
  summarise(avg_score = mean(score))
```

```
## # A tibble: 3 x 2
##   name    avg_score
##   <chr>    <dbl>
## 1 Alice      82.5
## 2 Bob        85
## 3 Charlie    95
```

We can achieve exactly the same results with chaining %>%. First we declare the dataframe, followed by grouping the rows, and then calculating the average score. You can continue to chain further operations. The advantage of this method is that it minimizes the need to nest functions. The latest version of R has ‘|>’ another piping method that works similarly, for age related reasons this tends to be less common and does have some differences

```
#Another example taking the mean of score via piping
df$score %>% mean()
```

```
## [1] 87.5
```

```
df$score |> mean()
```

```
## [1] 87.5
```

```
df$score %>% mean() %>% round() #dplyr
```

```
## [1] 88
```

```

df$score |> mean() |> round() #New R

## [1] 88

round(mean(df$score)) #Traditional programming with nested functions

## [1] 88

round(3.14159, digits = 2) #round takes two arguments the value to round and digits

## [1] 3.14

#both methods default to piping the first argument
3.14159 %>% round(digits=2)

## [1] 3.14

3.14159 |> round(digits=2)

## [1] 3.14

2 %>% round(3.14159, digits = .) #the stop specifies a position

## [1] 3.14

2 |> round(3.14159, digits = _) #the underscore specifies the positions

## [1] 3.14

#mutate becomes useful for creating or modifying columns in a pipeline.
x<-c(1,2,3)
y<-sin(x)
dt<- data.frame(x = x, y = y)
dt$newcol <- cos(dt$x) #Base style
dt <- dt %>%
  #uncomment to explore
  #newcol <- cos(x)
  #df$newcol <- cos(df$x) #You will notice that <- causes an error in the pipeline
  #{dt$newcol <- cos(dt$x); dt} # you can still use this with {}, it is a bit convoluted
  mutate(newcol = cos(x)) #Common method for changing columns in pipelines
dt

##      x      y    newcol
## 1 1 0.8414710 0.5403023
## 2 2 0.9092974 -0.4161468
## 3 3 0.1411200 -0.9899925

```

Importing

```
file_id <- "1ymbNqfv9s6YGZzN93HFKAhjg0Z5xZXV1"
url <- paste0("https://drive.google.com/uc?id=", file_id)

df <- read.csv(url)
```

1. Create a daily totals dataframe

Your original data (df) has one row per activity/run. But we want a new dataframe with one row per day, showing the total distance run that day.

Hint:

Using mutate or otherwise create a new dataframe called daily with:

date: the calendar day (YYYY-MM-DD)

day_km: total kilometres run on that day

%>% chain or otherwise into a group by to split the data by day

%>% chain again or otherwise into a **summarise** of the information, aggregating on the sum of distance.

```
## # A tibble: 6 x 2
##   date      day_km
##   <date>    <dbl>
## 1 2011-01-05    2.13
## 2 2011-01-10    2.20
## 3 2011-01-17    2.24
## 4 2011-01-21    2.40
## 5 2011-01-27    2.28
## 6 2011-02-02    2.42
```

2. Calculate running 365 day km total.

Next, calculate the running total distance over the previous 365 days. You should note that not every day has a run, and may first need to allocate 0km to these missing days. Note there are multiple methods of doing this, your method is less important than your understanding of a method. I will outline steps to my method;

Step 1:

Consider creating a full list of days in a vector between **min()** and **max()** of the days in the data set using **seq()** 'by = "day"'.

```
##           date
## 1 2011-01-05
## 2 2011-01-06
## 3 2011-01-07
## 4 2011-01-08
## 5 2011-01-09
## 6 2011-01-10
```

Step 2:

Compare the complete list of dates and adding zero to the missing ones. Start with `left_join(daily, by = "date")`. Chain into `mutate(day_km = ifelse(is.na(day_km), 0, day_km))` ie set the value to zero if not a number. Followed by `arrange`

<https://dplyr.tidyverse.org/reference/mutate-joins.html> <https://dplyr.tidyverse.org/reference/arrange.html>

```
##           date day_km
## 1 2011-01-05 2.1306
## 2 2011-01-06 0.0000
## 3 2011-01-07 0.0000
## 4 2011-01-08 0.0000
## 5 2011-01-09 0.0000
## 6 2011-01-10 2.2050
```

Step 3:

Calculate the cumulative sum for the last 365 days.

Start by mutating `daily` to contain:

`cumsum()` or the cumulative sum of the distance (we will call `cs`).

`trailing_365km = cs - dplyr::lag(cs, 365)` ie the cumulative sum - the cumulative sum from 365 days ago.

NOTE: Im printing `tail()` now not `head` as the first 364 days are NA.

$$\text{trailing_365}(t) = cs(t) - cs(t - 365)$$

<https://dplyr.tidyverse.org/reference/lead-lag.html>

```
##           date  day_km      cs trailing_365km
## 5307 2025-07-16 16.7450 6925.641      1561.579
## 5308 2025-07-17  0.0000 6925.641      1561.579
## 5309 2025-07-18  0.0000 6925.641      1552.896
## 5310 2025-07-19 16.7186 6942.359      1561.865
## 5311 2025-07-20  0.0000 6942.359      1545.277
## 5312 2025-07-21 17.4040 6959.763      1562.681
```

3. Lastly save the CSV.